

Chứng minh Tối ưu: Cosine Similarity → Inner Product qua L2-Normalization

Tài liệu trình bày nền tảng toán học, chứng minh hình thức, và phân tích hiệu năng phần cứng cho kỹ thuật tối ưu được áp dụng trong hệ thống DentalAI: thay thế phép đo Cosine Similarity bằng Inner Product trên không gian vector đã chuẩn hóa.

Tham chiếu mã nguồn:

- `src/retriever/engines.py` dòng 191, 204: `normalize_embeddings=True`
- `src/retriever/ingest.py` dòng 125: `faiss.IndexFlatIP(actual_dim)`

Phần 1: Nền tảng Toán học

1.1. Định nghĩa Cosine Similarity

Cho hai vector $\mathbf{Q}$ (query) và $\mathbf{D}$ (document) trong không gian $\mathbb{R}^n$ (với $n = 768$ cho vietnamese-sbert, $n = 1536$ cho text-embedding-3-small), **Độ tương đồng Cosine** được định nghĩa:

$$\text{CosineSim}(\mathbf{Q}, \mathbf{D}) = \frac{\mathbf{Q} \cdot \mathbf{D}}{|\mathbf{Q}| \times |\mathbf{D}|}$$

Trong đó:

- **Tử số** — Tích vô hướng (Dot Product / Inner Product):

$$\mathbf{Q} \cdot \mathbf{D} = \sum_{i=1}^n q_i \times d_i$$

- **Mẫu số** — Tích của hai Chuẩn L2 (Euclidean Norm):

$$|\mathbf{Q}| \times |\mathbf{D}| = \sqrt{\sum_{i=1}^n q_i^2} \times \sqrt{\sum_{i=1}^n d_i^2}$$

1.2. Chuẩn L2 (Euclidean Norm) là gì?

Về mặt đại số: Chuẩn L2 của vector $\mathbf{Q} = (q_1, q_2, \dots, q_n)$ được định nghĩa:

$$|\mathbf{Q}| = \sqrt{\sum_{i=1}^n q_i^2} = \sqrt{q_1^2 + q_2^2 + \dots + q_n^2}$$

Về mặt hình học: Chuẩn L2 chính là **độ dài** (magnitude) của vector trong không gian Euclid. Trong không gian 2 chiều, đây là khoảng cách từ gốc tọa độ $(0,0)$ đến đầu mút vector (q_1, q_2) , theo định lý Pythagore:

$$|\mathbf{Q}| = \sqrt{q_1^2 + q_2^2}$$

Khái quát lên n chiều, Chuẩn L2 đo "khoảng cách" từ gốc tọa độ đến điểm biểu diễn vector trong không gian $\mathbb{R}^n$.

1.3. Ý nghĩa hình học của Cosine Similarity

Cosine Similarity đo **cosine của góc** θ giữa hai vector:

$$\cos(\theta) = \frac{\mathbf{Q} \cdot \mathbf{D}}{|\mathbf{Q}| \times |\mathbf{D}|}$$

- $\cos(\theta) = 1$: hai vector cùng hướng (hoàn toàn tương đồng)
- $\cos(\theta) = 0$: hai vector vuông góc (không liên quan)
- $\cos(\theta) = -1$: hai vector ngược hướng (đối nghịch)

Đặc điểm quan trọng: Cosine Similarity **không phụ thuộc vào độ dài** vector, chỉ phụ thuộc vào **hướng**. Đây là tính chất then chốt cho bài toán tìm kiếm ngữ nghĩa — hai tài liệu nói cùng chủ đề nhưng khác độ dài vẫn có Cosine Similarity cao.

Phần 2: Chứng minh Tối ưu Toán học

2.1. Mệnh đề

Nếu mọi vector $\mathbf{Q}$ và $\mathbf{D}$ đều được L2-Normalize trước khi lưu trữ và truy vấn, thì:

$$\text{CosineSim}(\mathbf{Q}, \mathbf{D}) \equiv \mathbf{Q} \cdot \mathbf{D}$$

tức Cosine Similarity trở thành đồng nhất với Inner Product (Tích vô hướng).

2.2. Chứng minh

Bước 1: Định nghĩa phép L2-Normalization

Cho vector $\mathbf{Q} \in \mathbb{R}^n$ bất kỳ, vector chuẩn hóa $\hat{\mathbf{Q}}$ được xác định bởi:

$$\hat{\mathbf{Q}} = \frac{\mathbf{Q}}{|\mathbf{Q}|}$$

Tức mỗi thành phần $\hat{q}_i = \frac{q_i}{|\mathbf{Q}|}$.

Bước 2: Chứng minh vector chuẩn hóa có Chuẩn L2 bằng 1

$$|\hat{\mathbf{Q}}| = \sqrt{\sum_{i=1}^n \hat{q}_i^2} = \sqrt{\sum_{i=1}^n \left(\frac{q_i}{|\mathbf{Q}|}\right)^2}$$

Đưa hằng số $|\mathbf{Q}|$ ra ngoài dấu tổng:

$$= \sqrt{\frac{1}{|\mathbf{Q}|^2} \sum_{i=1}^n q_i^2} = \frac{1}{|\mathbf{Q}|} \sqrt{\sum_{i=1}^n q_i^2}$$

Nhận thấy $\sqrt{\sum_{i=1}^n q_i^2} = |\mathbf{Q}|$ theo định nghĩa Chuẩn L2, nên:

$$= \frac{1}{|\mathbf{Q}|} \times |\mathbf{Q}| = 1$$

$$\boxed{|\hat{\mathbf{Q}}| = 1 \quad \forall \mathbf{Q} \neq \mathbf{0}}$$

Kết luận: Sau L2-Normalization, mọi vector đều nằm trên **mặt cầu đơn vị** (unit hypersphere) trong $\mathbb{R}^n$. Tương tự, $|\hat{\mathbf{D}}| = 1$.

Bước 3: Thế vào công thức Cosine Similarity

$$\text{CosineSim}(\hat{\mathbf{Q}}, \hat{\mathbf{D}}) = \frac{\hat{\mathbf{Q}} \cdot \hat{\mathbf{D}}}{|\hat{\mathbf{Q}}| \times |\hat{\mathbf{D}}|}$$

Thay $|\hat{\mathbf{Q}}| = 1$ và $|\hat{\mathbf{D}}| = 1$:

$$= \frac{\hat{\mathbf{Q}} \cdot \hat{\mathbf{D}}}{1 \times 1} = \hat{\mathbf{Q}} \cdot \hat{\mathbf{D}}$$

$$\boxed{\text{CosineSim}(\hat{\mathbf{Q}}, \hat{\mathbf{D}}) = \hat{\mathbf{Q}} \cdot \hat{\mathbf{D}} = \sum_{i=1}^n \hat{q}_i \times \hat{d}_i}$$

Điều phải chứng minh (Q.E.D.): Trên không gian vector đã L2-Normalize, Cosine Similarity đồng nhất với Inner Product.

2.3. Hệ quả kiến trúc

Chứng minh trên cho phép thay thế:

Trước tối ưu	Sau tối ưu
<code>faiss.IndexFlatL2</code> + tính Cosine riêng	<code>normalize_embeddings=True</code> + <code>faiss.IndexFlatIP</code>
Mỗi query: $3n + 2\sqrt{}$ phép tính	Mỗi query: $2n$ phép tính (chỉ nhân-cộng)
Phép chia và khai căn mỗi lần truy vấn	Phép chia và khai căn chỉ xảy ra 1 lần khi ingest

Chi phí normalize được **khấu hao** (amortized): mỗi vector chỉ normalize 1 lần khi nhập vào hệ thống, nhưng được truy vấn hàng nghìn lần — tiết kiệm tích lũy rất lớn.

Phần 3: Tối ưu Kiến trúc Phần cứng (Hardware Optimization)

3.1. Phân tích chi phí phép tính

Xét hai vector n chiều. So sánh chi phí tính toán giữa Cosine gốc và Inner Product:

Cosine Similarity gốc (không normalize trước):

Phép tính	Số lượng	Chi phí CPU
Nhân $q_i \times d_i$ (tử số)	n phép nhân	n cycles
Cộng tích lũy tử số	$n-1$ phép cộng	$n-1$ cycles
Nhân q_i^2 (chuẩn Q)	n phép nhân	n cycles
Cộng tích lũy q_i^2	$n-1$ phép cộng	$n-1$ cycles
Nhân d_i^2 (chuẩn D)	n phép nhân	n cycles
Cộng tích lũy d_i^2	$n-1$ phép cộng	$n-1$ cycles
Khai căn $\sqrt{\sum q_i^2}$	1 phép SQRTSS	11-14 cycles
Khai căn $\sqrt{\sum d_i^2}$	1 phép SQRTSS	11-14 cycles
Nhân hai chuẩn	1 phép nhân	1 cycle

Phép tính	Số lượng	Chi phí CPU
Chia tử/mẫu	1 phép DIVSS	13-15 cycles

Tổng xấp xỉ: $6n + 35$ phép tính cơ bản.

Inner Product (sau L2-Normalize):

Phép tính	Số lượng	Chi phí CPU
Nhân $\hat{q}_i \times \hat{d}_i$	n phép nhân	n cycles
Cộng tích lũy	$n-1$ phép cộng	$n-1$ cycles

Tổng xấp xỉ: $2n$ phép tính cơ bản.

3.2. Tỷ lệ tăng tốc

Với $n = 768$ (vietnamese-sbert):

$$\text{Speedup} \approx \frac{6 \times 768 + 35}{2 \times 768} = \frac{4643}{1536} \approx 3.02 \times$$

Với $n = 1536$ (text-embedding-3-small):

$$\text{Speedup} \approx \frac{6 \times 1536 + 35}{2 \times 1536} = \frac{9251}{3072} \approx 3.01 \times$$

Tăng tốc khoảng **3 lần** cho mỗi cặp so sánh query-document.

3.3. Tại sao khai căn và chia chậm?

Trên kiến trúc x86-64 hiện đại (Intel/AMD), các phép tính có chi phí rất khác nhau:

Phép tính	Lệnh Assembly	Latency (cycles)	Throughput
Nhân số thực	MULSS / FMA	3-5	1/cycle
Cộng số thực	ADDSS / FMA	3-5	1/cycle
Khai căn	SQRTSS	11-14	1/3-7 cycles
Chia số thực	DIVSS	13-15	1/3-5 cycles

Lý do gốc rễ: Phép **nhân** và **cộng** là phép tính tuyến tính, mạch phần cứng (ALU) thực hiện trực tiếp trong pipeline. Phép **khai căn** và **chia** là phép tính phi tuyến, yêu cầu thuật toán lặp (iterative algorithm) bên trong bộ xử lý — bản chất là một vòng lặp Newton-Raphson ở mức mạch điện.

3.4. Tận dụng SIMD và FMA

Inner Product (chuỗi nhân-cộng) là ứng cử viên hoàn hảo cho:

- SIMD (Single Instruction, Multiple Data):** Lệnh **AVX-512** xử lý 16 phép nhân float32 trong 1 cycle. Với $n = 768$, chỉ cần $768 / 16 = 48$ vòng lặp SIMD.
- FMA (Fused Multiply-Add):** Lệnh **VFMADD231PS** thực hiện $a \times b + c$ trong 1 instruction duy nhất, hợp nhất phép nhân và cộng tích lũy — giảm 50% số instruction so với nhân rồi cộng riêng.

Cosine gốc không tận dụng được FMA hiệu quả vì phép khai căn và chia phá vỡ pipeline SIMD, tạo ra **pipeline stall** — bộ xử lý phải đợi kết quả khai căn xong mới thực hiện phép chia.

3.5. Tóm tắt: Vì sao kiến trúc này tối ưu

[Giai đoạn Ingest – chạy 1 lần duy nhất]

`normalize_embeddings=True`

- Mỗi vector qua: n phép nhân + n phép cộng + 1 SQRT + 1 DIV
- Chi phí: $O(n)$ cho mỗi document, tổng $O(N \times n)$ cho N documents
- Chỉ chạy khi xây dựng index, KHÔNG chạy khi truy vấn

[Giai đoạn Search – chạy mỗi lần người dùng hỏi]

IndexFlatIP: chỉ tính Inner Product

- Mỗi cặp (query, doc): n FMA instructions
 - Không có SQRT, không có DIV
 - SIMD-friendly, pipeline không bị stall
 - Nhân với $762 \text{ documents} \times 3 \text{ queries} = 2286$ lần tính/request
 - Tiết kiệm: $2286 \times (2 \text{ SQRT} + 1 \text{ DIV}) = 6858$ phép tính nặng bị loại bỏ
-